PUC-SP



Helipad Detector

Detecção de Helipontos em Imagens de Satélite de São Paulo com YOLO

Relatório Completo e Explicativo do Projeto

Pipeline técnico, metodologia, resultados e guia em linguagem simples

Pontifícia Universidade Católica de São Paulo (PUC-SP) — FACEI

Graduação em Human Centered-AI & Data Science

Machine Learning / Visão Computacional — Projeto P2

Professor: Rooney Ribeiro Albuquerque Coelho

Autor: ***Fabiana*** ⚡**Campaner**

São Paulo · 2026

Sumário

Este relatório existe em duas camadas ao mesmo tempo. Para quem já trabalha com dados e inteligência artificial, ele é um relatório técnico completo: cobre o pipeline de doze etapas, os três experimentos de treinamento, as métricas de avaliação e a validação de campo em dez bairros de São Paulo. Para quem nunca ouviu falar de YOLO ou de mAP, cada seção também traz uma explicação em linguagem simples — como se estivéssemos conversando sobre o projeto, sem pressupor conhecimento prévio de Inteligência Artificial.

Índice

- 1. O que é o Helipad Detector, em uma frase
- 2. Por que detectar helipontos — e por que isso é difícil
- 3. Como uma IA aprende a “ver” — explicação para leigos
- 4. Fonte de dados e escopo geográfico
- 5. O pipeline técnico completo — as 12 etapas
- 6. Coleta de imagens e anotação
- 7. Modelagem com YOLO — os quatro experimentos
- 8. Avaliação quantitativa — o que os números significam
- 9. Validação de campo — testando em dados reais, dez bairros
- 10. Análise qualitativa — acertos, erros e por quê
- 11. Aplicação web interativa
- 12. Pontos fortes, limitações e próximos passos
- 13. Ética, LGPD e governança
- 14. Conclusão
- 15. Glossário de termos técnicos
- 16. Camada extra — Tráfego de Helicóptero ao Vivo (Beta)

1. O que é o Helipad Detector, em uma frase

Em uma frase: é um sistema de Inteligência Artificial que olha para imagens de satélite de São Paulo e aponta, sozinho, onde existem helipontos nos telhados dos prédios.

Em termos um pouco mais técnicos: o Helipoint Detector é um pipeline completo de Detecção de Objetos que localiza helipontos em telhados na cidade de São Paulo, usando imagens aéreas/orbitais de alta resolução e modelos da família YOLOv8/YOLOv11. O projeto cobre todo o ciclo de vida de um sistema aplicado de visão computacional: coleta programática de imagens, automação geoespacial, criação e curadoria de um dataset próprio, anotação, pré-processamento, treinamento, avaliação quantitativa e qualitativa, inferência em bairros não vistos durante o treino, uma etapa extra de validação de campo em dados reais em dez bairros, e uma aplicação web interativa para demonstração.

> *Um ponto importante para quem não é da área: o projeto não usou nenhum banco de imagens pronto. Toda a base de dados — as fotos de satélite e as marcações de onde ficam os helipontos — foi construída do zero pela equipe, etapa que costuma consumir a maior parte do esforço em qualquer projeto sério de Inteligência Artificial.*

2. Por que detectar helipontos — e por que isso é difícil

Helipontos foram escolhidos como alvo porque, vistos de cima, eles têm um desenho bem característico: um grande “H” pintado sobre uma área circular ou quadrada no telhado. Isso parece fácil de reconhecer — mas na prática não é, e é justamente essa dificuldade que torna o projeto interessante do ponto de vista educacional.

Em imagens de satélite de uma cidade densa como São Paulo, o modelo pode se confundir com:

- Piscinas de prédios, que têm formato e contraste parecidos com o “H”;
- Quadras esportivas, com linhas e marcações geométricas semelhantes;
- Estruturas de telhado, caixas d’água e equipamentos de ar-condicionado;
- Superfícies reflexivas, que mudam de aparência dependendo da luz do dia.

> Esse tipo de confusão é chamado, na área, de falso positivo — quando o modelo “vê” um heliponto onde não existe nenhum. Trabalhar com um alvo que gera falsos positivos plausíveis é o que torna esse projeto um bom exercício de visão computacional: obriga a equipe a pensar sobre qualidade de anotação, diversidade de exemplos e os limites reais do modelo, em vez de comemorar um número bonito sem entender o que está por trás dele.

3. Como uma IA aprende a “ver” - explicação em linguagem simples

Antes de entrar no pipeline técnico, vale explicar, em termos simples, o que está acontecendo por baixo do capô.

O que é “Detecção de Objetos”?

É a tarefa de IA que responde a duas perguntas ao mesmo tempo, para cada imagem: “existe o objeto qu eu procuro aqui?” e “onde exatamente ele está?”. A resposta da segunda pergunta vem na forma de uma caixa retangular ao redor do objeto — chamada de bounding box (caixa delimitadora) — desenhada sobre a imagem.

O que é o YOLO?

YOLO é a sigla de “You Only Look Once” (“você só olha uma vez”), o nome de uma família de modelos de Inteligência Artificial muito usada para detecção de objetos em tempo real. Ele examina a imagem inteira de uma só vez e devolve, diretamente, a lista de objetos encontrados com suas respectivas caixas e um grau de confiança (de 0 a 100%) para cada detecção. Este projeto usa a versão YOLOv8n (a letra “n” indica a variante “nano”, mais leve e rápida de treinar) e também testa a variante mais recente YOLOv11n.

Como o modelo “aprende”?

O processo se chama treinamento. A equipe mostra ao modelo centenas de imagens já marcadas manualmente — “aqui tem um heliponto, e ele está exatamente nesta posição” — e o modelo ajusta, aos poucos, seus parâmetros internos para acertar cada vez mais essas marcações. Cada “passada” completa por todas as imagens de treino é chamada de época (epoch). Neste projeto, os modelos foram treinados por 60 a 100 épocas.

Por que a qualidade dos dados importa mais do que o modelo em si?

Essa é talvez a lição central do projeto: cerca de 80% do esforço em um projeto real de Inteligência Artificial está na construção e curadoria dos dados, não em ajustes finos de arquitetura do modelo. O YOLO usado por este grupo é essencialmente o mesmo YOLO que qualquer outra equipe usaria; o que diferencia o resultado final é a qualidade das imagens coletadas, a consistência das marcações e a diversidade geográfica do dataset.

4. Fonte de dados e escopo geográfico

O escopo geográfico segue o briefing acadêmico do curso: a cidade de São Paulo, com foco em bairros próximos a corredores corporativos densos, onde a presença de helipontos é mais comum — Perdizes, Higienópolis, Pacaembu, Sumaré, Avenida Paulista, Itaim Bibi, Pinheiros, Faria Lima, Berrini, Vila Olímpia, Brooklin e Morumbi.

De onde vêm as imagens?

- ESRI World Imagery (tiles XYZ) — fonte principal, com resolução sub-métrica e acesso HTTP programático (ou seja, pode ser baixada automaticamente por código);
- Google Earth Web — fonte complementar, usada apenas para capturas pontuais de alvos específicos, nunca para coleta em massa;
- GeoSampa — mencionada como fonte alternativa de alta resolução, um possível extra além do escopo básico.

> Sempre que uma imagem ou mosaico derivado do ESRI é reproduzido, o projeto inclui a atribuição obrigatória: “Source: Esri, Maxar, Earthstar Geographics, and the GIS User Community”.

5. O pipeline técnico completo — as 12 etapas

O pipeline completo, do zero à validação em campo, pode ser resumido em doze etapas. Cada uma delas está descrita abaixo com o nome técnico e uma explicação em linguagem simples do que ela significa na prática.

1 e 2 — Descoberta e extração de registros de helipontos

Um robô de automação (Selenium, em src/geospatial/helipad_bot.py) navega por um site público de aviação, procurando registros de helipontos e extraindo suas coordenadas geográficas e metadados. Em linguagem simples: em vez de procurar manualmente, uma no navegador, “onde existem helipontos no Brasil”, um programa faz essa busca sozinho, de forma sistemática.

3 — Organização das coordenadas

Os dados coletados são salvos e organizados em src/geospatial/helipad_coordinates.csv, uma planilha simples com data, coordenadas e o bairro correspondente.

4 — Conversão em bounding boxes geográficas

O script src/geospatial/transform_coordinates.py converte cada coordenada de um heliponto (um ponto único) em uma pequena área retangular ao redor dele — a bounding box geográfica —, que será usada para decidir qual pedaço do mapa baixar.

5 — Download de imagens de satélite

Para cada bounding box, o sistema baixa os “tiles” (pedaços quadrados de imagem de satélite) do ESRI World Imagery, no zoom 19 — o nível de aproximação recomendado para alvos pequenos como helipontos.

6 — Construção de mosaicos

Os tiles baixados são unidos em mosaicos maiores, organizados por bairro e por zoom (notebook src/geospatial/geospatial_image_collection.ipynb), formando imagens contínuas prontas para inspeção visual.

7 — Triagem manual

Uma pessoa da equipe revisa os mosaicos e descarta os pedaços de imagem que claramente não contêm heliponto, mantendo apenas o material relevante para anotação — evitando desperdiçar tempo anotando imagens vazias.

8 e 9 — Anotação e exportação do dataset

As imagens selecionadas sobem para o Roboflow, plataforma onde a equipe desenha anotações (majoritariamente polígonos, convertidos em caixas delimitadoras no treino de detecção) ao redor de cada heliponto, usando uma única classe (“heliponto”). Em seguida, o Roboflow aplica pré-processamento (redimensionamento para 640×640 pixels) e divide o conjunto em treino, validação e teste (178/34/14 imagens na versão 2), exportando tudo em formato compatível com o YOLO. A versão 2 do dataset não usa aumento de dados; só o dataset com fork do exp3 usa.

10 — Treinamento

Os modelos YOLOv8n são treinados no Google Colab, usando uma GPU T4 gratuita, monitorando métricas e curvas de aprendizado ao longo das épocas.

11 — Inferência em bairro não visto

O modelo treinado é testado em um bairro que nunca apareceu durante o treinamento, para avaliar sua capacidade de generalização — ou seja, se ele realmente “aprendeu” o padrão visual de um heliponto, e não apenas decorou as imagens de treino.

12 — Validação de campo em dez bairros

Além do teste padrão, o melhor modelo foi rodado contra 7.943 tiles reais de satélite, cobrindo dez bairros inteiros de São Paulo — muito além do conjunto curado de treino/validação/teste, aproximando o projeto de um cenário real de uso. Das 225 imagens anotadas com coordenadas, 125 são cópias de tiles de campo, então esta etapa não é totalmente independente do conjunto anotado (ver Seção 9).

6. Coleta de imagens e anotação

6.1 Coleta programática

A coleta segue o padrão de tiles XYZ do ESRI World Imagery: zoom 19 para alvos pequenos como helipontos; bounding boxes definidas por bairro; conversão para índices de tile por meio de uma função chamada deg2tile; download com verificação do status HTTP e filtragem de imagens “placeholder” (imagens em branco que indicam falha); e organização das imagens em pastas por bairro e nível de zoom.

6.2 Curadoria e volume do dataset

- Volume mínimo de 200 imagens com o objeto-alvo após a curadoria;
- Diversidade geográfica: pelo menos 3 bairros diferentes no treino;
- Reserva de pelo menos 1 bairro totalmente não visto para o teste final de generalização;
- Triagem manual dos tiles, descartando recortes sem heliponto.

6.3 Anotação no Roboflow

O Roboflow foi usado como plataforma central de anotação e gestão do dataset — não como origem das imagens, apenas como ferramenta de marcação, versionamento, pré-processamento e exportação. As regras seguidas pela equipe foram:

- Classe única: heliponto;
- Caixas delimitadoras justas (bem ajustadas ao objeto, sem sobra de área);
- Critérios escritos para casos ambíguos — objetos parcialmente visíveis, sombras, reflexos;
- Trabalho de anotação dividido entre os membros da equipe, para evitar viés de um único anotador.

6.4 Pré-processamento e divisão do dataset

- Redimensionamento para 640×640 pixels;
- Aumento de dados: não usado na versão 2 do dataset; só o dataset com fork do exp3 o aplica (rotação de ±15°, variação de brilho de 25%, espelhamentos horizontal/vertical);
- Divisão na versão 2: 178 imagens de treino, 34 de validação e 14 de teste (226 no total, cerca de 79/15/6%).

> Cada arquivo de anotação (.txt) contém, por linha, coordenadas normalizadas no formato (classe, x_centro, y_centro, largura, altura) — o formato padrão esperado pelo YOLO. Na versão 2, 170 dos 178 arquivos de rótulo do treino e todos os de validação e teste são polígonos, que o treino de detecção converte em caixas.

7. Modelagem com YOLO — os três experimentos

O treinamento foi feito com Ultralytics YOLOv8n no Google Colab (GPU T4), usando Python, PyTorch e a biblioteca roboflow para integração direta do dataset. Seguindo o briefing do curso, que pede pelo menos dois experimentos variando um hiperparâmetro por vez, a equipe rodou quatro:

- exp1 — dataset original, 60 épocas, imgsz 640, batch 16, seed 42. Execução de Pedro Vyctor Almeida (pasta exp1-2) na versão 2 do dataset; o vínculo com a versão 2 é evidência documental (args.yaml e pesos), sem contagem de imagens registrada. Ultralytics 8.4.75.
- exp2 — dataset original, 100 épocas (mesmos demais parâmetros), para testar se treinar por mais tempo melhora o resultado ou causa overfitting (quando o modelo "decora" o treino em vez de aprender o padrão geral). Ultralytics 8.4.98; autor e ambiente diferentes do exp1, então exp1 × exp2 não é uma ablação controlada.
- exp3 — versão aumentada (fork) do dataset, com rotação de ±15°, variação de brilho de 25% e espelhamento horizontal/vertical, também por 100 épocas — mantido como um ponto de comparação separado por usar uma versão diferente do dataset. Ultralytics 8.4.98 (mesma versão do exp2).

- exp4 — dataset original (mesmo do exp1/exp2), 100 épocas, mesmo batch (16), imgsz (640) e seed (42) do exp2 — mudando apenas o modelo-base: YOLO11n (yolo11n.pt) no lugar do YOLOv8n. Rodado especificamente para comparar as duas arquiteturas na mesma configuração, item que já estava listado como melhoria futura sugerida neste relatório. O Ultralytics 8.4.163 sinaliza três arquivos de rótulo do treino que misturam polígono e caixa (o exp2 usou o 8.4.98, sem sinalização).

7.1 Um problema de integridade de dados — encontrado e corrigido

Durante o desenvolvimento, a equipe identificou um problema importante: uma execução inicial do notebook havia baixado silenciosamente o dataset com fork (o dataset aumentado), mas salvado os resultados sob o nome "exp2", o que gerou documentação incorreta sobre qual dataset havia sido realmente usado naquele experimento.

> *Esse tipo de erro é conhecido como um problema de integridade de dados: o código rodou sem travar, os números pareciam plausíveis, mas a atribuição do experimento estava errada. Em qualquer projeto real de IA, esse é exatamente o tipo de falha silenciosa mais perigosa — porque não gera um erro visível, apenas uma conclusão falsa.*

A correção foi em duas partes: renomear a execução equivocada para exp3 (já que, na prática, era o experimento com o dataset aumentado) e re-rodar o exp2 corretamente, contra o workspace do dataset original. Como salvaguarda contra recorrência, foi adicionado um guard de assert no notebook de treino — uma verificação automática que interrompe a execução caso o dataset carregado não seja o esperado para aquele experimento. O registro do exp3 conta 356/34 imagens (fork); o do exp2 refeito, 178/34 (original).

8. Avaliação quantitativa — o que os números significam

Antes das tabelas, um pequeno glossário das quatro métricas usadas para avaliar o modelo, explicadas em linguagem simples:

- Precision (Precisão): de todas as vezes que o modelo disse “achei um heliponto”, quantas dessas vezes ele estava certo? Precisão alta significa poucos alarmes falsos.
- Recall (Revocação): de todos os helipontos que realmente existem nas imagens, quantos o modelo conseguiu encontrar? Recall alto significa que o modelo raramente deixa passar um heliponto real.
- mAP@50: uma nota única que resume a qualidade geral das detecções, considerando um critério “razoável” de sobreposição entre a caixa prevista e a caixa real (50% de sobreposição mínima).
- mAP@50-95: a mesma ideia, mas com um critério muito mais rigoroso, exigindo sobreposição quase perfeita em vários níveis — o padrão usado em benchmarks internacionais como o COCO.

8.1 Resultados dos três experimentos

Os três experimentos são descobertos automaticamente e comparados ao vivo na aba “Experiment Metrics” do dashboard Streamlit do projeto. Estes são os resultados na melhor época de cada execução:

Experimento	Melhor Época	Precision	Recall	mAP@50	mAP@50-95
exp1 (60 ép., dataset original)	54	1,000	0,963	0,994	0,881
Experimento	**Melhor Época**	**Precision**	**Recall**	**mAP@50**	**mAP@50-95**
exp2 (100 ép., dataset original)	81	0,982	0,971	0,993	0,888
exp3 (100 ép., fork aumentado)	78	0,943	1,000	0,993	0,885
exp4 (100 ép., dataset original, YOLO11n)	54	0,970	0,948	0,993	0,877

Leitura em linguagem simples desses resultados: o exp2 supera levemente o exp1 em mAP@50-95 (uma diferença de +0,0065) ao treinar por mais tempo (100 em vez de 60 épocas), enquanto o exp1 permanece marginalmente à frente em Precision na sua melhor época. Essas diferenças são pequenas o suficiente para estarem dentro da margem de ruído esperada, dado que o dataset é pequeno (226 imagens ao todo) — ou seja, treinar mais não trouxe um ganho dramático, mas também não piorou o modelo. A comparação não é controlada: autor, ambiente e versão do Ultralytics (8.4.75 e 8.4.98) diferem, então a diferença não deve ser atribuída apenas às épocas.

Um sinal importante de saúde do treinamento: as curvas de loss (a "taxa de erro" do modelo durante o aprendizado) não mostram overfitting em nenhuma das três execuções — a loss de validação acompanha de perto a loss de treino ao longo de todas as épocas, em vez de se distanciar dela. Isso indica que o modelo está aprendendo um padrão genuíno de "o que é um heliponto", e não apenas memorizando as imagens de treino.

8.2 exp2 vs. exp4 — YOLOv8n vs. YOLO11n (mesma base, mesmas épocas)

exp2 vs. exp4 — YOLOv8n vs. YOLO11n (mesma base, mesmas épocas)

Diferente do exp3 (que muda o dataset), o exp4 muda apenas a arquitetura: mesmo dataset original do exp2, mesmas 100 épocas, mesmo batch (16), imgsz (640) e seed (42) — trocando só o modelo-base, de YOLOv8n para YOLO11n (yolo11n.pt). É a única comparação deste relatório isolando puramente a arquitetura, sem nenhuma outra variável mudando junto.

- Precision (melhor época): exp2 0,982 vs. exp4 0,970
- Recall (melhor época): exp2 0,971 vs. exp4 0,948
- mAP@50 (melhor época): exp2 0,993 vs. exp4 0,993 — empate
- mAP@50-95 (melhor época): exp2 0,888 vs. exp4 0,877

Leitura honesta: nesta configuração e neste dataset pequeno (226 imagens), o YOLO11n não superou o YOLOv8n em nenhuma das quatro métricas na melhor época — ficou marginalmente atrás em Precision, Recall e mAP@50-95, e empatado em mAP@50. A diferença é pequena o suficiente para estar dentro do ruído esperado de um dataset deste tamanho, então a conclusão correta não é "YOLO11n é pior", e sim "nesta escala de dados, a arquitetura mais nova não trouxe ganho mensurável" — um resultado tão válido de se documentar quanto um ganho teria sido.

Limitação importante: quando este relatório foi escrito, o exp4 ainda não havia passado pela validação de campo (Seção 9) nem pela comparação de generalização por bairro ("Comparando exp1, exp2, exp3 e exp4") — essa validação foi concluída depois, e os resultados já estão incorporados na Seção 9 e na comparação acima.

Este experimento resolve o item “comparar sistematicamente YOLOv8n vs. YOLOv11n”, listado como melhoria futura sugerida na Seção 12.3 deste relatório.

8.3 Gráficos de treinamento — exp1 (60 épocas, dataset original)

Estes gráficos foram gerados diretamente a partir do results.csv real do exp1, o experimento-base do projeto

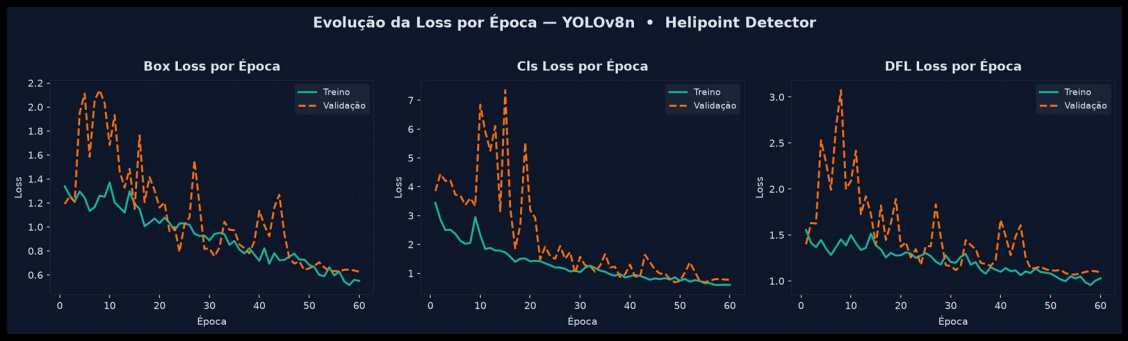


Figura 1 — Evolução das losses (Box, Cls, DFL) por época, treino vs. validação. Sem sinal evidente de overfitting nas 60 épocas; val/cls_loss oscila nas primeiras 20 épocas e estabiliza a partir da 30.



Figura 2 — Precision e Recall ao longo do treino. Precision estabiliza acima de 0,95 a partir da época 30; Recall alcança 0,97+ nas épocas finais, após uma queda acentuada por volta da época 5.

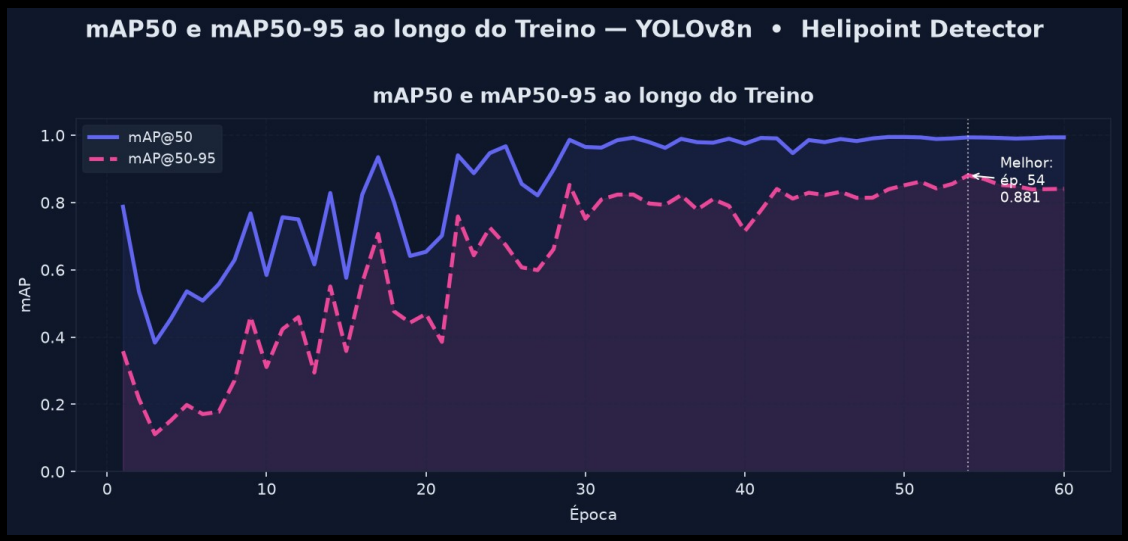
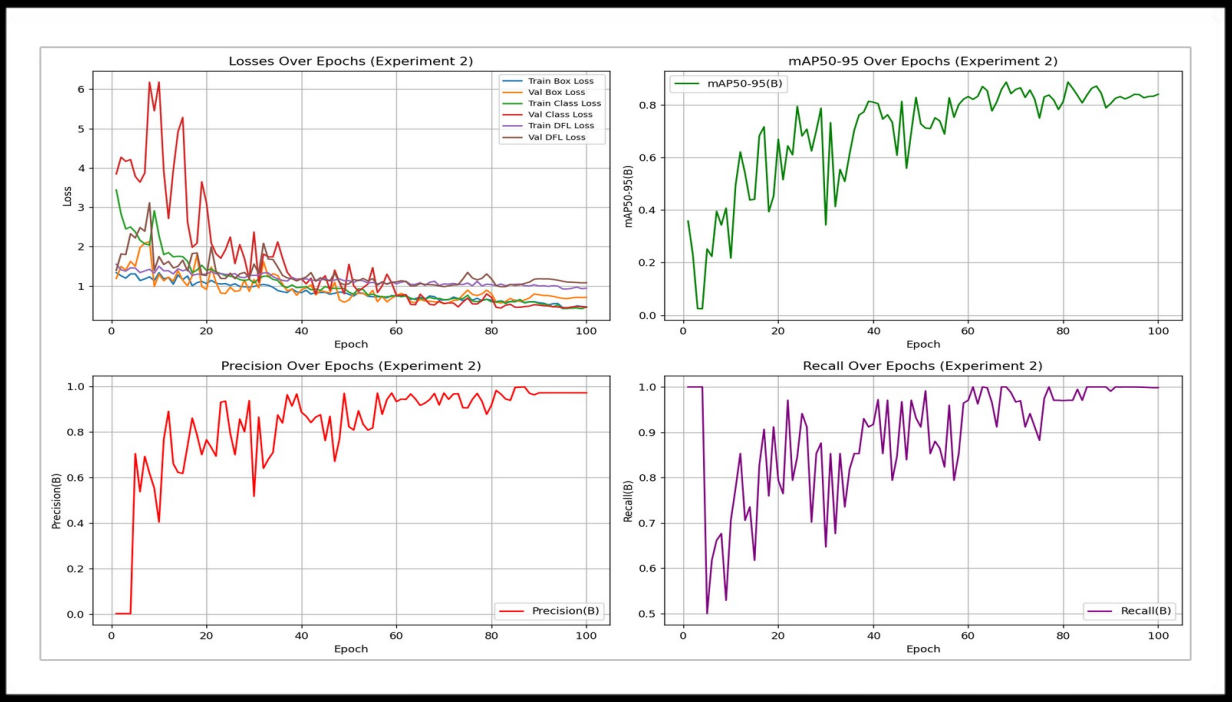


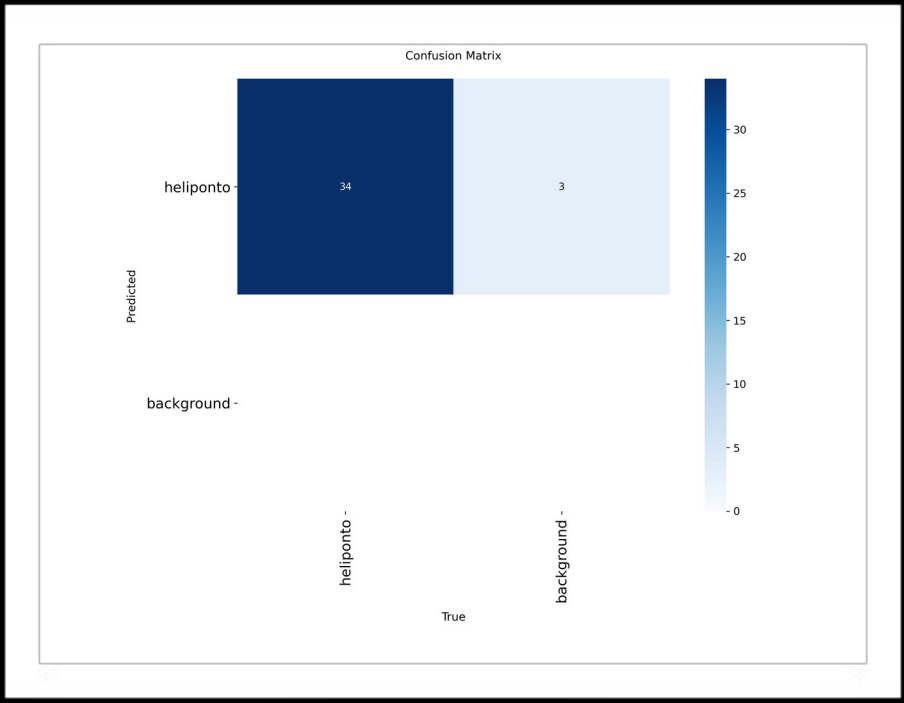
Figura 3 — mAP@50 e mAP@50-95. Pico na melhor época (54), após o qual as métricas flutuam levemente.

8.4 Gráficos de treinamento — exp2 (100 épocas, dataset original)

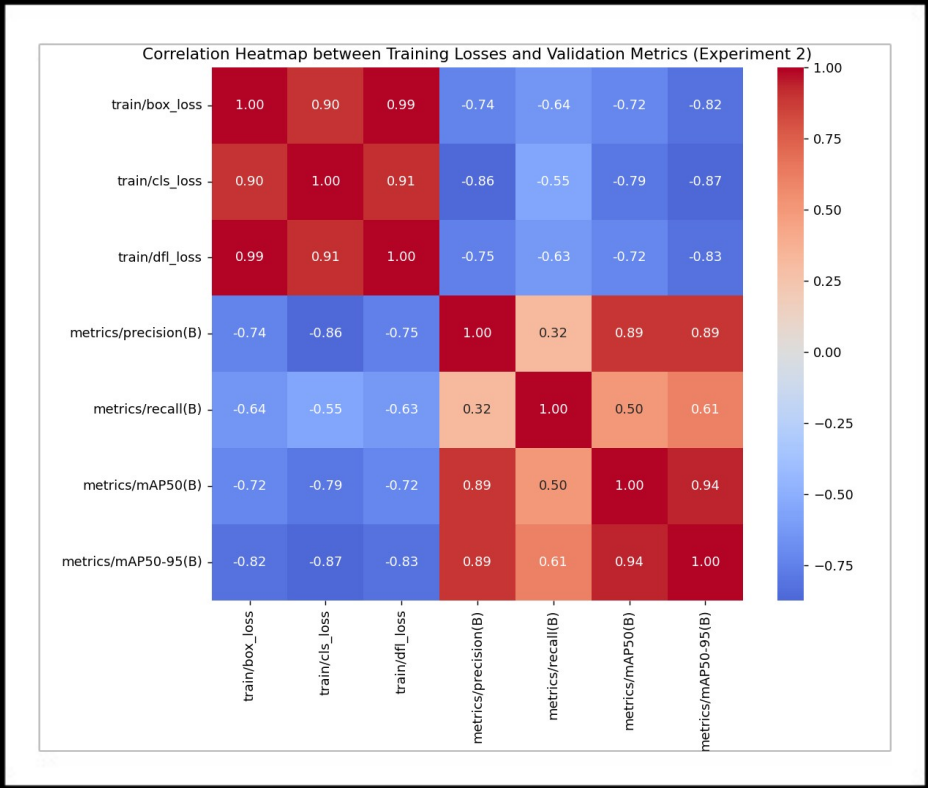
O exp2 é a comparação que o briefing pede (60 → 100 épocas) sobre o mesmo conteúdo de dados (versão 2), mas não é uma ablação controlada: autor, ambiente e versão do Ultralytics (8.4.75 e 8.4.98) diferem.



Visão geral — losses, mAP@50-95, Precision e Recall ao longo de 100 épocas (exp2).



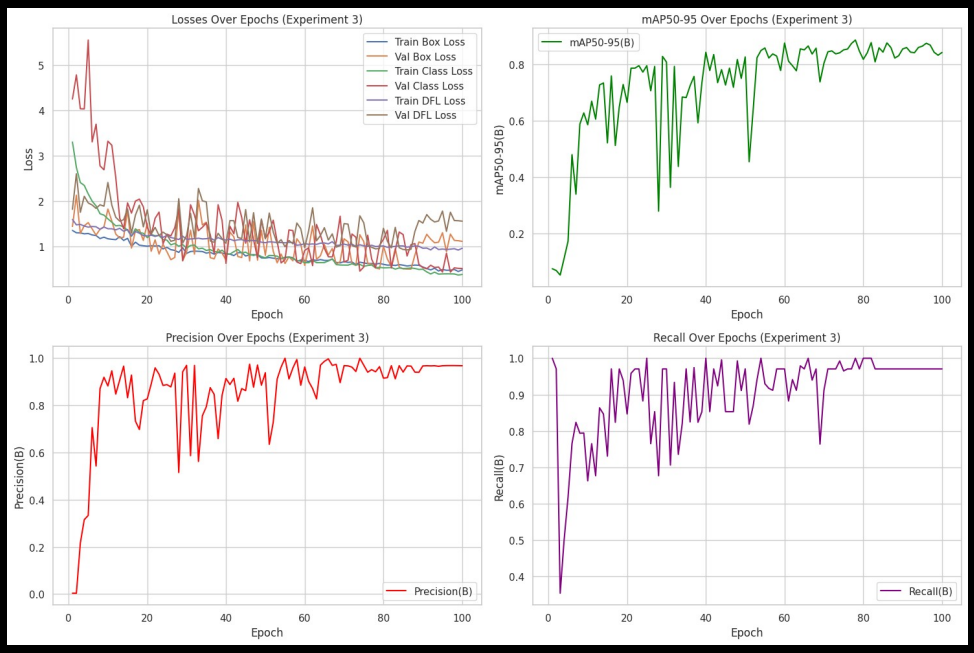
Matriz de confusão (validação, exp2): 34 acertos, 3 falsos positivos de fundo confundido com heliponto.



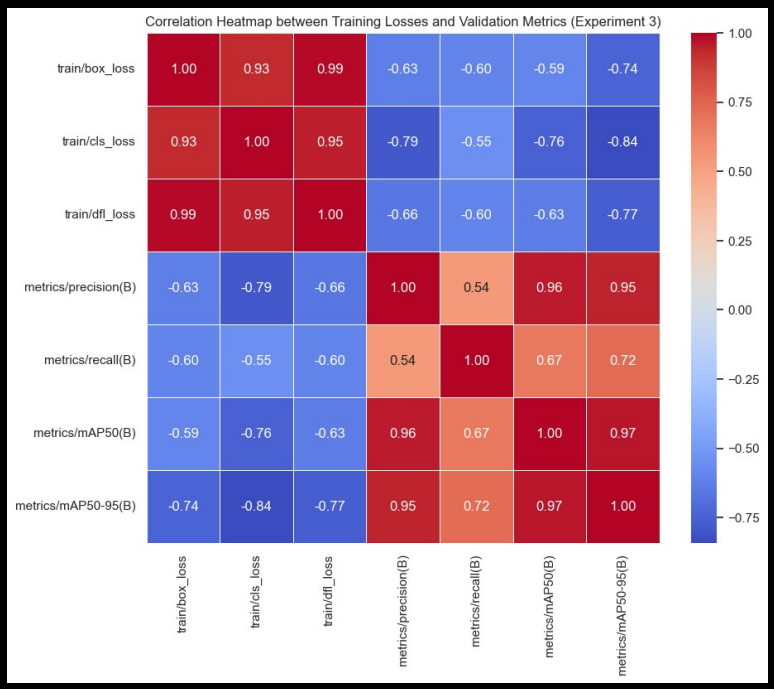
Correlação entre as losses de treino e as métricas de validação (exp2).

8.5 Gráficos de treinamento — exp3 (100 épocas, dataset com fork/augmentation)

O exp3 usa uma versão diferente do dataset (fork com augmentation extra), por isso funciona como ponto de referência adicional, e não como comparação direta de ablação — essa comparação é a exp1 vs. exp2, acima.



Visão geral — losses, mAP@50-95, Precision e Recall ao longo de 100 épocas (exp3). Mesmo padrão de leve queda entre a melhor época e a final observado no exp1.



Correlação entre as losses de treino e as métricas de validação (exp3): cls_loss é a mais fortemente (anti)correlacionada com mAP@50-95 (-0,84).

9. Validação de campo — testando em dados reais, dez bairros

Um diferencial deste projeto em relação ao mínimo exigido pelo briefing é a etapa de validação de campo: em vez de avaliar o modelo apenas no conjunto de teste curado (imagens já selecionadas manualmente), o melhor modelo (exp2) foi rodado contra 7.943 tiles reais de satélite, sem nenhuma curadoria prévia, cobrindo dez bairros inteiros de São Paulo — incluindo corredores corporativos densos como Itaim Bibi, Avenida Paulista, Vila Olímpia, Pinheiros, Brooklin e Faria Lima. Ressalva: 125

das 225 imagens anotadas com coordenadas (87/177 no treino, 25/34 na validação, 13/14 no teste) são cópias de tiles de campo — 2,3% dos 5.540 tiles distintos (7.767 pares região × tile no conjunto de comparação) — então esta etapa não é totalmente independente do conjunto anotado. Detalhes em reports/lineage_audit/.

Em termos simples: em vez de mostrar ao modelo só as “melhores fotos”, a equipe o soltou para trabalhar em um bairro inteiro, tile por tile, exatamente como ele funcionaria em um uso real.

Os tiles e bounding boxes por região vêm de src/geospatial/sp_neighborhoods_bbox.csv; o download de tiles e a inferência são automatizados pelos scripts src/geospatial/download_all_regions.py e auto_triage_regions.py, ambos retomáveis — ou seja, o processo pode ser pausado e retomado depois sem reprocessar o que já foi feito, algo essencial ao lidar com milhares de imagens.

Bairro	Detectados	Total de Tiles	Taxa
Inter-Zone Corridor	133	480	27,7%
Itaim Bibi	191	750	25,5%
Brooklin	231	1.014	22,8%
Vila Olímpia	158	660	23,9%
Av. Paulista (Trecho 1)	179	840	21,3%
Faria Lima	170	840	20,2%
Av. Paulista (Trecho 2)	157	780	20,1%
Pinheiros	244	1.232	19,8%
Vila Nova Conceição	109	572	19,1%
Alphaville Industrial	105	775	13,6%

TOTAL	**1.677**	**7.943**	**21,1%**

Interpretação dos resultados: Inter-Zone Corridor e Itaim Bibi apresentam as maiores taxas de detecção, o que é coerente com serem corredores corporativos densos — regiões com muitos prédios comerciais e, portanto, mais chances reais de haver helipontos. Alphaville Industrial, com perfil mais industrial e menos torres corporativas, apresenta a menor taxa, alinhada ao esperado para esse tipo de uso do solo. Esse tipo de coerência entre o resultado do modelo e o conhecimento urbano da cidade é

um bom sinal de que o modelo está captando um padrão real, e não gerando detecções aleatórias.

9.1 Comparando exp1, exp2, exp3 e exp4 na mesma validação de campo

A tabela de validação de campo da Seção 9 usa a execução original do exp2, sobre um conjunto de 7.943 tiles. O exp1 e o exp3 foram rodados sobre um conjunto ligeiramente menor, de 7.767 tiles (o Faria Lima e a divisão da Av. Paulista diferem em algumas centenas de tiles), a diferença é pequena o suficiente para não afetar a comparação abaixo, mas este relatório afirmava antes que os três usavam exatamente os mesmos 7.943 tiles, o que estava incorreto. Para comparar o exp4 (YOLO11n) de forma justa com os outros três, o exp2 foi rodado de novo sobre esse mesmo conjunto de 7.767 tiles. O total dessa nova execução (21,0%) ficou praticamente idêntico ao da execução original de 7.943 tiles (21,1%) — confirmação independente de que a diferença de tiles não é o que explica as conclusões abaixo.

Bairro	exp1	exp3	exp2 (repet.)	exp4
Alphaville Industrial	6,1%	13,3%	13,6%	15,6%
Av. Paulista (Trecho 1)	9,8%	26,8%	21,3%	21,7%
Av. Paulista (Trecho 2)	9,3%	23,8%	20,1%	20,2%
Brooklin	9,7%	14,2%	22,8%	15,0%
Faria Lima	9,0%	17,2%	18,7%	16,1%
Inter-Zone Corridor	14,2%	20,8%	27,7%	22,9%
Itaim Bibi	11,9%	17,9%	25,5%	20,9%
Pinheiros	8,2%	16,6%	19,8%	15,8%
Vila Olímpia	12,9%	16,5%	23,9%	21,2%
Vila Nova Conceição	8,0%	12,1%	19,1%	17,7%
TOTAL	**9,6%**	**17,9%**	**21,0%**	**18,3%**

Achado principal:

o exp1 lidera o conjunto de validação curado em Precision (1,000, ver Seção 8.1) — mas, nas mesmas condições de campo, detecta cerca de metade dos helipontos reais que o exp2 (9,6% vs. 21,0%–21,1%, dependendo do conjunto de tiles) e sensivelmente menos que o exp3 (17,9%). O modelo com a melhor nota num benchmark pequeno e curado não é automaticamente o que generaliza melhor em

cobertura real de satélite sem filtragem prévia; exatamente o tipo de lacuna que a validação de campo em larga escala, além do split padrão de treino/validação/teste, existe para capturar.

Esse resultado também reforça a decisão de usar o exp2 em produção, em vez do exp1 (que tinha a Precision mais alta no papel). No benchmark curado, o exp4 ficou praticamente empatado com o exp3 (mAP@50-95 0,877 vs. 0,885); em campo, o exp4 fica um pouco melhor que o exp3 (18,3% vs. 17,9%), mas os dois ficam bem atrás do exp2 (21,0%) — a direção de uma comparação no benchmark curado entre dois modelos não necessariamente se mantém em campo, a mesma lição que o achado acima já mostrava.

Detecção visual automatizada baseada em imagem, sujeita a erros e não equivalente a validação operacional, legal, de segurança ou aeronáutica.

10. Análise qualitativa — acertos, erros e por quê

Números resumidos, como Precision ou mAP, contam apenas parte da história. Por isso, tanto as imagens curadas de teste quanto a rodada de validação de campo do Faria Lima foram revisadas caso a caso, olhando manualmente para o que o modelo acertou e o que ele errou:

Tipo	Tile de Exemplo	Confiança	Observação
Acerto claro	tile_z19_x194126_y297485.jpg	0,94	Padrão “H” nítido em quadrado bem definido no telhado
Acerto claro	tile_z19_x194143_y297481.jpg	0,96	Geometria e contraste claros, caixa bem ajustada
Acerto desafiador	tile_z19_x194545_y298183.jpg	0,77	Detecção correta mesmo com ângulo/iluminação menos favorável
Falso Positivo	tile_z19_x194129_y297480.jpg	0,78	Caixa sobre quadra esportiva, não heliponto
Falso Positivo	tile_z19_x194547_y298176.jpg	0,86	Caixa sobre piscina, geometricamente parecida com o “H”
Falha de dado	tile_z19_x194139_y297467.jpg	—	Tile vazio/preto (falha de download do ESRI) marcado como “Detected”

Dois padrões de erro merecem destaque, porque ambos são explicáveis — e explicáveis é uma palavra-chave em avaliação de IA responsável: um erro que a equipe entende e consegue justificar é muito mais tratável do que um erro opaco.

- Falsos positivos previsíveis: piscinas e quadras esportivas compartilham geometria e contraste retangular semelhantes ao "H" do heliponto — esse é um erro de confusão visual, tratável adicionando mais exemplos negativos (imagens de piscinas e quadras marcadas como "não é heliponto") em treinos futuros.

- Tiles vazios marcados como detecção: esse não é um erro do modelo, e sim uma falha do pipeline de dados — quando o download de um tile falha parcialmente, ele pode ficar preto ou vazio, e o modelo às vezes reage a esse ruído. A correção recomendada, já documentada para a próxima iteração do projeto, é checar o desvio-padrão de pixel da imagem antes de rodar a inferência, descartando automaticamente tiles vazios.

10.1 Exemplos reais de detecção (conjunto de teste curado, exp1)

Abaixo, recortes reais gerados pelo próprio modelo (exp1), extraídos de reports/model_outputs/detect/predict/, mostrando a caixa prevista e o grau de confiança da detecção — o mesmo tipo de saída que aparece na aplicação web.

Acertos claros (alta confiança)



heliponto — confiança 0,95. Padrão "H" nítido, caixa bem ajustada ao contorno do heliponto.



heliponto — confiança 0,86. Marcação clara identificada mesmo com vegetação e sombra próximas ao telhado.

Acerto desafiador



heliponto — confiança 0,64. Confiança moderada; a estrutura do heliponto é menos evidente visualmente nesta captura.

Falsos positivos



heliponto — confiança 0,28. Marcação de solo/via confundida com padrão de heliponto — confiança baixa, coerente com um falso positivo.



heliponto — confiança 0,28. Outro caso de via/pavimento gerando falso positivo, novamente com confiança baixa.

> Nota metodológica: nenhum exemplo de falso negativo confirmado foi incluído nesta versão do relatório, os tiles sem detecção disponíveis não têm anotação de referência (ground truth) que confirme a presença real de um heliponto não detectado. Adicionar isso exige cruzar com o rótulo original do dataset antes de publicar a alegação, para não arriscar uma afirmação incorreta.

11. Aplicação web interativa

O arquivo apps/streamlit_app/app.py implementa um dashboard Streamlit completo com nove abas, indo bem além do requisito opcional do briefing de uma simples interface de demonstração:

Aba	Finalidade
Experiment Metrics	Descobre automaticamente cada experimento treinado, comparando Precision/Recall/mAP ao vivo
Field Detections by Region	Cards, gráfico de barras e tabela da validação de campo em 10 bairros
Map	Mapa Folium em modo escuro com 3 camadas alternáveis, incluindo uma
Aba	**Finalidade**
	camada de taxa de detecção em escala de azul
Search by Region	Busca ao vivo por bounding box: baixa tiles e roda inferência sob demanda
Sample Images	Detecções reais pré-selecionadas para demonstração instantânea
Upload Image	Upload de qualquer imagem aérea/satélite para detecção anotada
Pipeline	Passo a passo visual do pipeline completo de 12 etapas
Governance	Declarações de justiça, LGPD e limitações conhecidas
Downloads	Relatórios, dataset, métricas e artefatos da validação de campo

Em linguagem simples: essa aplicação é a “vitrine” do projeto, permite que qualquer pessoa, mesmo sem saber programar, faça upload de uma foto de satélite, busque uma região do mapa, ou simplesmente veja os resultados já processados, sem precisar rodar nenhum código.

12. Pontos fortes, limitações e próximos passos

12.1 Pontos fortes

- Cobertura de ponta a ponta do ciclo de vida de um projeto de visão computacional — da coleta bruta de dados até uma aplicação demonstrável;
- Foco explícito em engenharia de dados, e não apenas em ajustar o modelo;
- Automação geoespacial que aumenta escala e reprodutibilidade da coleta;
- Uma etapa de validação de campo em dados reais, além dos benchmarks curados — algo incomum em projetos acadêmicos deste porte;
- Uma camada de aplicação interativa completa, com nove abas funcionais.

12.2 Limitações observadas

- Confusão visual com piscinas e quadras esportivas continua sendo a principal fonte de falsos positivos;
- Os resultados dependem da consistência de anotação entre os membros da equipe;
- A capacidade de generalização é limitada pela diversidade de padrões de telhado vistos durante o treino;
- Falhas de download de tiles (tiles vazios) podem gerar detecções espúrias na validação de campo.

12.3 Melhorias futuras sugeridas

- Adicionar mais exemplos negativos (piscinas, quadras esportivas) para reduzir falsos positivos;
- Adicionar um filtro automático de tile vazio antes da inferência no pipeline de validação de campo;
- Estender a validação de campo para outros bairros e para a região totalmente não vista da Baixada Santista;
- Incorporar ciclos de active learning, re-anotando os exemplos mais difíceis identificados na validação de campo.

13. Ética, LGPD e governança

O projeto segue práticas-chave de IA responsável, resumidas aqui tanto para o leitor técnico quanto para quem está avaliando o projeto do ponto de vista de conformidade:

- Uso exclusivo de imagens de satélite de área pública;
- Nenhuma anotação de pessoas, placas de veículos ou outros identificadores pessoais;
- IA generativa usada apenas como ferramenta de apoio, com uso documentado no relatório;
- Foco acadêmico e de pesquisa, sem qualquer intenção de vigilância individual.

Na perspectiva da LGPD (Lei Geral de Proteção de Dados), o projeto minimiza dados pessoais identificáveis, documenta claramente as fontes de imagem e sua atribuição obrigatória (Esri, Maxar, Earthstar Geographics e a GIS User Community), e restringe o uso dos resultados a fins acadêmicos e de pesquisa.

13.1 Incidente de segurança identificado e corrigido

Durante o desenvolvimento, a equipe identificou e corrigiu um problema real de segurança: uma chave de API do Roboflow estava hardcoded (escrita diretamente em texto puro) nos notebooks de treino. A chave foi removida do código-fonte, substituída por leitura via variável de ambiente / Colab Secrets, e marcada para revogação.

> *Em linguagem simples: uma "senha" de acesso ao Roboflow estava visível para qualquer pessoa que abrisse o notebook — inclusive em um repositório público no GitHub. Isso é um risco real, porque quem visse essa chave poderia usá-la para acessar o workspace do projeto. A equipe corrigiu o problema movendo a chave para fora do código, um exemplo concreto de por que boas práticas de gestão de segredos (secrets management) importam mesmo em projetos acadêmicos, e não só em produtos comerciais.*

Como prática de governança de IA, o repositório prioriza: fontes de dados controladas, critérios de anotação claros e documentados, reprodutibilidade dos experimentos via notebooks e seeds fixas, avaliação de erros estruturada (a análise qualitativa da Seção 10) e documentação explícita das decisões de design tomadas ao longo do pipeline — incluindo o próprio erro de integridade de dados descrito na Seção 7.1, mantido no relatório de forma transparente em vez de omitido.

14. Conclusão

O Helipad Detector atinge métricas de nível profissional — mAP@50 de aproximadamente 99% e mAP@50-95 de aproximadamente 88% — em um dataset construído inteiramente do zero pela equipe.

A etapa de validação de campo confirma que esses resultados se sustentam também em cobertura real de satélite, sem curadoria prévia, em dez bairros de São Paulo, com padrões de erro documentados e explicáveis, em vez de falhas opacas.

Isso valida tanto o modelo em si quanto a ênfase mais ampla do projeto: qualidade de dado, governança de anotação, reprodutibilidade e documentação honesta e baseada em evidências — inclusive quando essas evidências mostram um erro cometido e corrigido pela própria equipe, como aconteceu na atribuição inicial do exp2/exp3.

Para quem chega até aqui sem experiência técnica em IA, a mensagem central deste relatório pode ser resumida assim: construir um bom sistema de Inteligência Artificial não é, na maior parte do tempo, um problema de algoritmo , é um problema de dados bem coletados, bem anotados, bem testados e bem documentados. Este projeto é, acima de tudo, um exemplo prático e transparente desse princípio.

15. Glossário de termos técnicos

Referência rápida dos termos técnicos usados ao longo deste relatório, para consulta a qualquer momento.

Termo	Significado em linguagem simples
Bounding box	Caixa retangular desenhada ao redor de um objeto em uma imagem, indicando onde ele está.
Dataset	Conjunto de dados (aqui, imagens + marcações) usado para treinar e testar o modelo.
Época (epoch)	Uma passagem completa do modelo por todas as imagens de treino.
Falso Negativo (FN)	Quando existe um heliponto real na imagem, mas o modelo não o detecta.
Falso Positivo (FP)	Quando o modelo detecta um heliponto onde, na verdade, não existe nenhum.
Generalização	Capacidade do modelo de funcionar bem em imagens novas, nunca vistas durante o treino.
IoU (Intersection over Union)	Medida de quanto a caixa prevista pelo modelo se sobrepõe à caixa real do objeto.
mAP (mean Average Precision)	Nota única que resume a qualidade geral das detecções do modelo, considerando vários limiares de sobreposição.
Overfitting	Quando o modelo “decora” os exemplos de treino em vez de aprender o padrão geral, funcionando mal em dados novos.
Precision (Precisão)	De todas as detecções feitas, quantas estavam corretas.
Recall (Revocação)	De todos os objetos reais existentes, quantos o modelo conseguiu encontrar.
Tile	Pedaço quadrado e padronizado de uma imagem de mapa/satélite, usado para montar mosaicos maiores.
YOLO	“You Only Look Once” — família de modelos de IA para detecção de objetos em tempo real.

16. Camada extra — Tráfego de Helicóptero ao Vivo (Beta)

Além do pipeline central de detecção por satélite, o dashboard inclui uma camada experimental, adicionada a partir de feedback do curso: uma sub-aba dentro de "Map" que mostra posições reais de aeronaves em voo perto de São Paulo, obtidas ao vivo de uma fonte pública de dados ADS-B (adsb.fi).

Em linguagem simples: além de "onde os telhados têm um heliponto" (o que o YOLO faz), essa camada extra mostra "quais aeronaves estão voando agora" — um tipo de dado completamente diferente, vindo de um radar público, não de imagem de satélite.

Essa camada é rotulada como **Beta** de forma permanente e deliberada — não é um aviso temporário a ser removido depois. Ela depende de um feed de terceiros e de uma heurística de classificação por melhor esforço, não do modelo ou do dataset deste projeto, e por isso segue um padrão de confiabilidade diferente do restante do dashboard.

16.1 Da onde vêm os dados, e por quê

A primeira fonte testada foi a OpenSky Network (gratuita, sem necessidade de chave de API). O endpoint público (`/states/all`) funcionava normalmente a partir de uma conexão doméstica, mas apresentava timeout de conexão TCP a partir de todo host de nuvem testado — tanto o Streamlit Community Cloud quanto o Hugging Face Spaces falhavam de forma idêntica. Isso é consistente com uma prática comum de redes voluntárias de ADS-B: bloquear faixas de IP de datacenter/nuvem como medida antiabuso.

Duas outras fontes gratuitas foram avaliadas e descartadas na mesma investigação: api.airplanes.live (retornava 403 em todos os endpoints, hoje exige chave) e api.adsb.lol (retornava sempre uma lista vazia de aeronaves). A fonte adotada, **adsb.fi**, continuava servindo dados reais sem exigir chave. A confirmação de que ela funcionava a partir de hospedagem em nuvem veio de inspecionar o projeto open-source **Osiris** (github.com/simplifaisoul/osiris, licença MIT) — um dashboard OSINT não relacionado a este projeto, que consome exatamente esse mesmo endpoint a partir de uma aplicação Next.js hospedada na Vercel, e que havia enfrentado e resolvido, de forma independente, os mesmos dois problemas descritos acima (bloqueio da OpenSky em nuvem e o campo category vazio).

Um ganho adicional do adsb.fi sobre a OpenSky: ele reporta um código de tipo de aeronave real (campo t, ex.: "R44", "EC35", "S76") para a maioria dos contatos, em vez do campo category da OpenSky — que a investigação do Osiris mediu como vazio em cerca de 96% das aeronaves, mesmo quando solicitado explicitamente. A própria abordagem de classificação por código de tipo, incluindo a lista de códigos usada (HELICOPTER_TYPE_CODES, em apps/streamlit_app/app.py), foi **adaptada da lógica equivalente do Osiris** — crédito registrado aqui, no README (seções "Fontes de Imagem e Texto" e "Escopo da Camada de Tráfego Aéreo ao Vivo") e como comentário no código-fonte (fetch_live_helicopter_flights()), pela mesma razão que este relatório documenta os próprios erros com transparência: uma solução técnica que veio de fora merece ser creditada, não apenas absorvida. A heurística de exclusão por prefixo de callsign de companhia aérea continua ativa, mas apenas como reserva, para os contatos que não reportam nenhum código de tipo.

16.2 Como a identificação de helicóptero funciona (e seus limites)

A maioria dos contatos reporta um código real de tipo de aeronave, comparado contra uma lista de códigos ICAO de helicópteros civis/executivos/de resgate plausíveis sobre São Paulo — quando há correspondência, a aeronave é marcada como **confirmada**. Para os contatos sem código de tipo, a camada recorre a uma heurística de exclusão por callsign: qualquer callsign que comece com um prefixo conhecido de companhia aérea comercial é descartado; tudo o mais é mantido como candidato **não confirmado** , o que pode incluir aviões pequenos, táxi aéreo ou jatos executivos, não apenas helicópteros.

A cobertura também é parcial por natureza: o feed só enxerga aeronaves dentro do alcance de um receptor ADS-B voluntário e que estejam de fato transmitindo no momento. Um mapa vazio ou com poucos pontos significa "nada detectado por este feed agora", nunca "nenhum helicóptero voando". Os rastros de voo exibidos são construídos localmente, na sessão do navegador, ponto a ponto, a partir dos momentos em que o dashboard esteve aberto — não é o histórico completo de rota da aeronave.

16.3 Um incidente de deploy identificado e corrigido

Durante a publicação desta camada no Streamlit Cloud, o deploy quebrou com uma falha de build associada ao Python 3.14 — a versão padrão detectada automaticamente pela plataforma para este repositório. A correção foi adicionar um arquivo runtime.txt fixando explicitamente a versão de Python usada em produção, eliminando a ambiguidade sobre qual interpretador o Streamlit Cloud deveria provisionar.

Em linguagem simples: o código funcionava no ambiente de desenvolvimento local, mas a nuvem tentou rodar tudo em uma versão de Python mais nova e ainda não totalmente compatível com as dependências do projeto. Fixar a versão em um arquivo de configuração resolveu isso — o mesmo tipo de disciplina de reprodutibilidade documentado neste relatório para outros incidentes reais (Seção 7.1, sobre a atribuição incorreta de dataset entre exp2/exp3; Seção 13.1, sobre a chave de API do Roboflow exposta).

16.4 Reformulação de apresentação (UX)

Após a funcionalidade estar em produção, a apresentação foi revisada para evitar dois problemas de comunicação com o usuário leigo:

- A camada estava misturada, como uma quarta opção de camada, dentro do mesmo mapa Folium da detecção de helipontos — o que podia sugerir, incorretamente, que o tráfego ao vivo de helicópteros fazia parte da mesma pipeline de visão computacional. Ela foi movida para uma sub-aba própria, com seu próprio mapa, dissociada visualmente do resultado central do projeto.

- O aviso técnico completo era exibido com st.warning() — um componente visual de fundo amarelo/oliva, que uma pessoa sem contexto tende a interpretar como "algo deu errado" em vez de "isto tem limitações conhecidas". O texto técnico completo passou a ficar dentro de um painel expansível, com apenas uma frase curta e neutra visível por padrão; foi adicionada também uma legenda visual simples (ícone sólido de helicóptero para "confirmado" vs. ícone com selo "?" para "não confirmado"), reduzindo a dependência de texto para transmitir a diferença entre os dois níveis de confiança.

16.5 Nota de responsabilidade

Como em todo o restante do projeto, uma posição exibida nesta camada — confirmada ou não — é telemetria ao vivo de terceiros, sujeita às limitações de cobertura e classificação descritas acima. Ela não é uma detecção do modelo YOLO deste projeto, não corresponde a nenhuma coordenada de heliponto do dataset, e não deve ser interpretada como validação operacional, legal, de segurança ou aeronáutica de qualquer heliponto detectado nas demais seções deste relatório.